

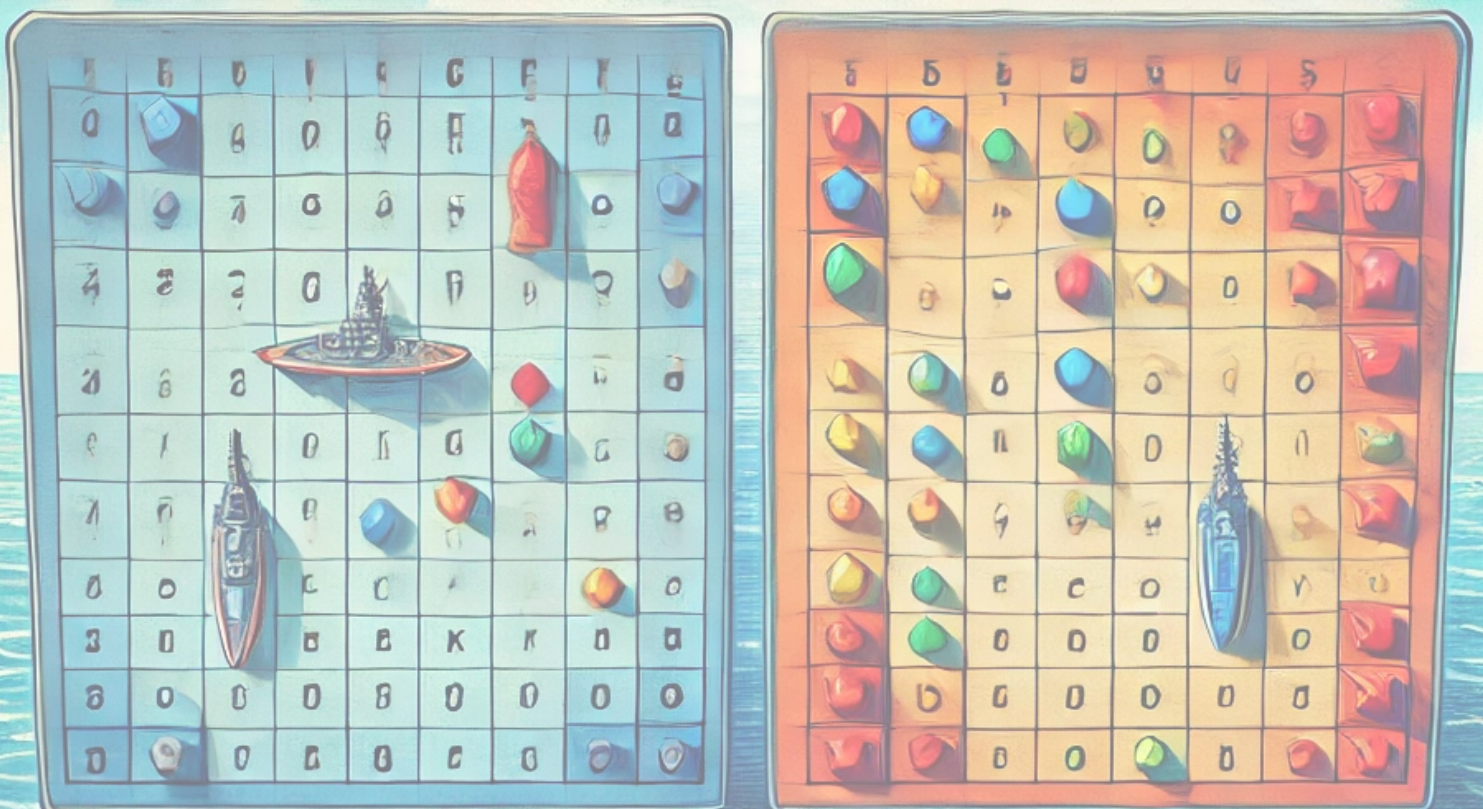
Universidad Aut3noma de Barcelona
Menci3n de Ingenier3a del Software
Test y Calidad del Software

Tocado y Hundido

Meritxell Argente Guirao - 1392328

Fiorella Lucia Queirolo Chavez - 1666868

Lunes 10:30 - 12:30



Índice

1. Introducción	3
2. Modelo	4
2.1. Clase Coordinada	4
2.2. Clase Casilla	5
2.3. Clase Agua	5
2.4. Clase Barco	6
2.5. Clase Tablero	7
2.5.1. Constructor	7
2.5.2. Colocación de los barcos	8
2.5.3. Recibir golpe	10
2.5.4. Comprobación barcos hundidos	10
2.6. Clase Jugador	11
2.6.1. Mock	12
2.7. Clase JugadorIA	12
2.7.1. Métodos generales	12
2.7.2. Colocación de los barcos	13
2.7.3. Gestión del golpe de las casillas	14
2.7.4. Mock	15
2.8. Clase JugadorPersona	15
2.8.1. Métodos generales	15
2.8.2. Colocación de barcos	16
2.9. Clase Partida	17
2.9.1. Mockito	18
3. Controlador	18
3.1. Mockito	19
3.2. Pairwise testing	19
4. Vista	21
5. Pruebas de caja blanca	21
5.1. Statement coverage	21
5.2. Decision coverage	22
5.3. Condition coverage	23
5.4. Path coverage	24
5.5. Loop Testing	25
6. CI/CD	26

1. Introducción

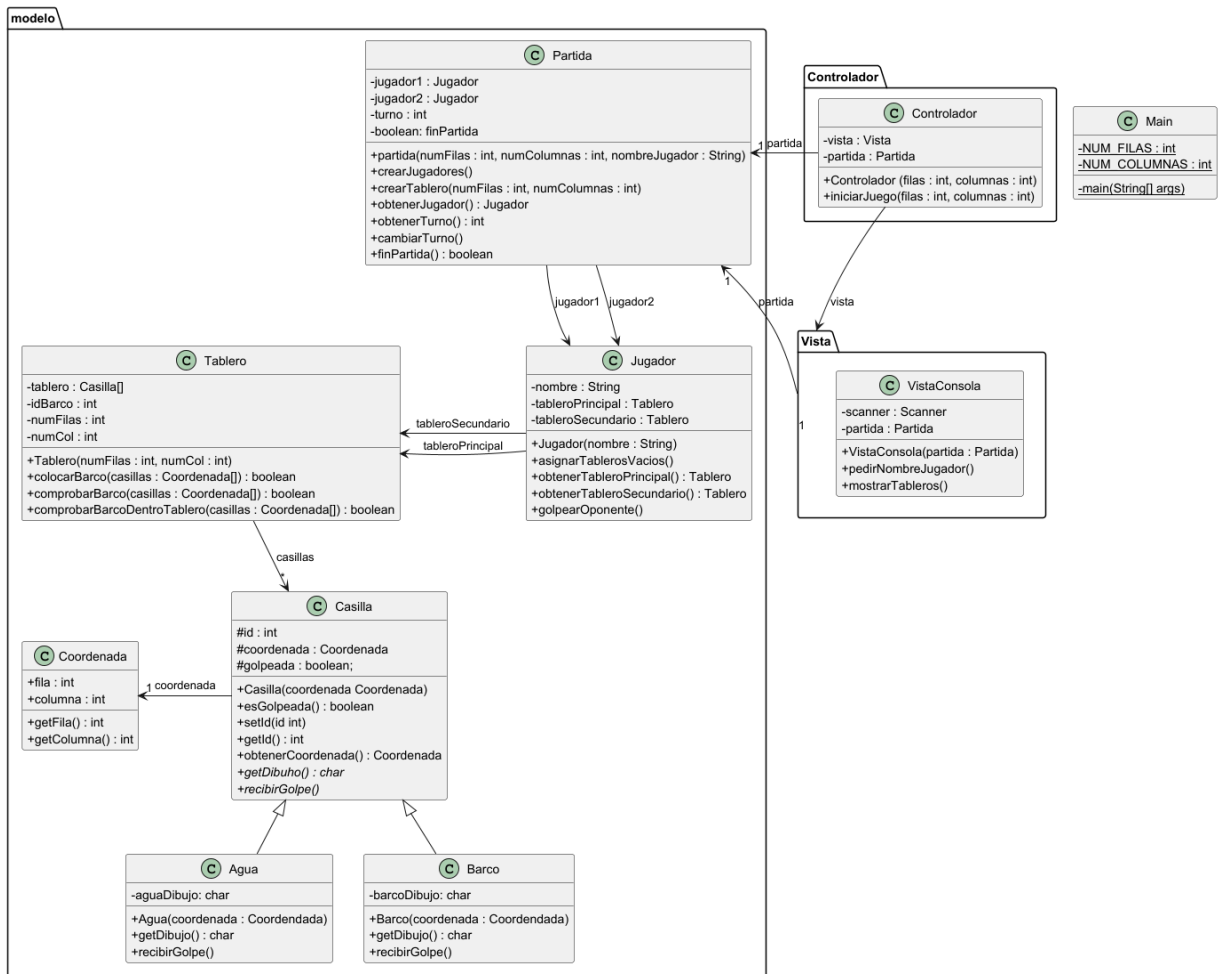
Para el desarrollo de esta práctica, hemos optado por implementar el juego de Tocado y Hundido. Este juego, también comúnmente conocido como Batalla Naval, trata de que dos jugadores deben colocar una flota de barcos en su tablero, concretamente colocan 5 barcos de diferentes dimensiones. Durante la partida irán indicando en qué coordenadas quieren lanzar un golpe tratando de derribar todos los barcos del oponente. El primer jugador que consiga derribarlos será el ganador de la partida.

Para que pueda entenderse la implementación del juego, debemos aclarar las reglas que hemos decidido:

- El tablero siempre será cuadricular. El tamaño mínimo será 10x10 y el máximo 15x15.
- Habrá un mínimo y máximo de 5 barcos a colocar en el tablero de las siguientes dimensiones:
 - 1 barco de 2 coordenadas
 - 2 barcos de 3 coordenadas
 - 1 barco de 4 coordenadas
 - 1 barco de 5 coordenadas
- El formato tanto para colocar los barcos como para golpear será en todo caso y poniendo un ejemplo "1A", siendo 1 la fila y A la columna. En caso de no introducir bien la coordenada se volverá a solicitar al jugador que introduzca la coordenada en el formato solicitado.
- En ningún caso podrá colocarse un barco en posición diagonal.

Para la implementación del juego se ha utilizado el lenguaje de programación Java en el entorno de desarrollo IntelliJ IDEA. Además, se ha seguido la metodología TDD (*Test Driven Development*), es decir, se han implementado las pruebas unitarias antes de la implementación del juego. Asimismo, se han realizado pruebas de caja blanca donde nos hemos asegurado de cubrir el código implementado. Tanto las pruebas de caja blanca como las de caja negra se explicaran con detalle para cada funcionalidad a lo largo de este informe. Este proyecto se ha ido desarrollando junto con la herramienta GitHub, se puede ver en el siguiente enlace [Tocado y Hundido](#).

Por último, indicar que en el inicio de este proyecto, antes de realizar ninguna prueba, se realizó el siguiente diagrama de clases que nos ayudó a tener una idea general sobre la implementación del juego.



2. Modelo

Las siguientes implementaciones que se van a mostrar corresponden a las clases de Modelo. Estas se encargan de la lógica interna del juego, teniendo todos los métodos necesarios para actualizar los dos diferentes tipos de casillas (agua y barco), crear y mantener el tablero correctamente a medida que avanza el juego, proporcionar las opciones para el jugador y especificar lo que hace la IA.

2.1. Clase Coordenada

Funcionalidad: Esta clase se encarga de determinar una coordenada en el tablero, con los atributos fila (int) y columna (int). Los métodos de esta clase son el constructor, los correspondientes getters y setters y un método equals() para poder comparar dos objetos de esta clase.

Localización:

- Archivo: Coordenada.java
- Clase: Coordenada
- Métodos:
 - Coordenada(int fila, int col)

- getFila()
- getCol()
- setCol(int col)
- setFila(int fila)
- equals(Object o)

Test:

- Archivo: CoordenadaTest.java
- Clase: CoordenadaTest
- Métodos:
 - testConstructor_expectedTrue()
 - testEquals_expectedTrue()
 - testEquals_expectedFalseRow()
 - testEquals_expectedFalseCol()

2.2. Clase Casilla

Funcionalidad: Esta clase representa una casilla dentro del tablero del juego, esta puede ser agua o un barco. Se trata de una clase abstracta con los atributos coordenada (Coordenada), golpeada (boolean) e id (int). Los métodos de esta clase son el constructor, los correspondientes getters y setters y los métodos abstractos getDibujo() y recibirGolpe().

Localización:

- Archivo: Casilla.java
- Clase: Casilla
- Métodos:
 - Casilla(int Coordenada)
 - obtenerCoordenada()
 - getId()
 - esGolpeada()
 - equals()
 - getDibujo()
 - recibirGolpe()

Test: Para esta clase no hemos realizado test unitarios, pues al ser una clase abstracta se puede instanciar directamente. No obstante, sus métodos son testeados en sus dos clases derivadas que se presentarán a continuación.

2.3. Clase Agua

Funcionalidad: Esta clase representa una casilla que corresponde al agua dentro del tablero. La clase Agua deriva de la clase Casilla. Su atributo, además de su clase base, es aguaDibujo(String) con el que representamos un objeto de esta clase en el tablero. Los métodos de esta clase son su constructor, getDibujo() y recibirGolpe().

Para esta clase hemos realizado, en primer lugar, tests unitarios. En dichos tests se ha comprobado que, cuando una casilla Agua recibe un golpe, cambie su dibujo con el que representamos este tipo de casilla en el tablero. Además, comprobamos que aunque se golpee varias veces la misma casilla (situación que no puede darse por la lógica del juego, pero quisimos testear), esta mantenía su atributo golpeado como true y no variaba su dibujo a no golpeada.

Cabe decir que se creó una primera versión en la rama Agua, la cual después de analizar mejor la estructura del juego se le hizo cambios en la segunda rama AguaV2.

Localización:

- Archivo: Agua.java
- Clase: Agua
- Métodos:
 - Agua(Coordenada coordenada)
 - getDibujo()
 - recibirGolpe()

Test:

- Archivo: AguaTest.java
- Clase: AguaTest
- Métodos:
 - testEsGolpeada_expectedFalse()
 - testRecibirGolpe_expectedTrue()
 - testRecibirGolpe_variosGolpes_expectedTrue()
 - testGetDibujo_sinGolpear_expectedTrue()
 - testGetDibujo_conGolpe_expectedTrue()
 - testEquals_mismoObjeto_expectedTrue()
 - testEquals_objetoNull_expectedFalse()
 - testEquals_objetoDeOtraClase_expectedFalse()

2.4. Clase Barco

Funcionalidad: Creación de la clase Barco que deriva de la clase Casilla. Su atributo, igual que la clase Agua, que también deriva de Casilla, es un dibujo (String) con el que representamos un Barco en el tablero. De la misma forma, sus métodos son getDibujo() y recibirGolpe().

Dado que esta clase, como la expuesta anteriormente, Agua, siguen la misma lógica, pues lo único que las diferencia, es el dibujo que las representa, los tests realizados son exactamente los mismos que los realizados en la clase Agua.

Cabe decir que se creó una primera versión en la rama Barco, la cual después de analizar mejor la estructura del juego se le hizo cambios en la segunda rama BarcoV2.

Localización:

- Archivo: Barco.java
- Clase: Barco
- Métodos:
 - Barco(Coordenada coordenada)
 - getDibujo()
 - recibirGolpe()

Test:

- Archivo: BarcoTest.java
- Clase: BarcoTest
- Métodos:
 - testEsGolpeada_expectedFalse()
 - testRecibirGolpe_expectedTrue()
 - testRecibirGolpe_variosGolpes_expectedTrue()
 - testGetDibujo_sinGolpear_expectedTrue()
 - testGetDibujo_conGolpe_expectedTrue()
 - testEquals_mismoObjeto_expectedTrue()
 - testEquals_objetoNull_expectedFalse()
 - testEquals_objetoDeOtraClase_expectedFalse()

2.5. Clase Tablero

Dado que tanto la clase Tablero como los tests realizados son muy extensos, dividiremos este apartado en bloques para su mejor entendimiento. Cabe decir que se creó una primera versión en la rama Tablero, la cual después de analizar mejor la estructura del juego se hizo cambios en la segunda rama TableroV2.

2.5.1. Constructor

Funcionalidad: Creación de la clase Tablero con los siguientes atributos: tablero (ArrayList<Casilla>), idBarco(int), numFilas(int), numCols(int).

Este apartado lo dedicamos únicamente a la lógica y tests del constructor. Cómo se ha explicado en la introducción de este informe, se da la opción al usuario de escoger el tamaño del tablero con el que desea jugar (siempre dentro de unos límites como ya se ha especificado).

Localización:

- Archivo: Tablero.java
- Clase: Tablero
- Métodos:
 - Tablero(int dimensión)
 - comprobarDimension(int dimension)

Test:

- Archivo: TableroTest.java
- Clase: TableroTest
- Métodos de Test asociado a la funcionalidad:
 - TestConstructorDimension10_expectedTrue()
 - TestConstructorDimension9_expectedTrue()
 - TestConstructorDimension15_expectedTrue()
 - TestConstructorDimension16_expectedTrue()
 - TestConstructorDimension12_expectedTrue()
 - TestConstructorDimension20_expectedTrue()
 - TestConstructorDimension11_expectedTrue()
 - TestConstructorDimension14_expectedTrue()
 - TestConstructorDimensionNegative_expectedTrue()
 - TestConstructorDimension0_expectedTrue()

Pese a que el tablero no es el encargado de verificar que el usuario introduce de forma correcta la dimensión del tablero deseada, hemos decidido que también debe asegurarse de, aunque reciba alguna dimensión del tablero por parámetro no deseada, debe construir este tal y como se ha especificado en las reglas del juego. Por ello, hemos realizado los tests de caja negra con las siguientes consideraciones:

- Particiones equivalentes:
 - Dimensión dentro del rango permitido (válido)
 - Dimensión inferior al mínimo permitido (no válido)
 - Dimensión superior al máximo permitido (no válido)
- Valores frontera:
 - Mínimo permitido: 10x10
 - Máximo permitido 15x15
- Valores límite:
 - 11x11 y 9x9
 - 14x14 y 16x16
- Casos adicionales para que sea más robusto:
 - Dimensión negativa
 - Dimensión 0
 - Dimensión 20

2.5.2. Colocación de los barcos

Funcionalidad: Colocación de los barcos en el tablero, ya sean del jugador o de la IA. Para esta funcionalidad también debemos tener presente las reglas presentadas en la introducción de este informe:

- Cantidad y dimensiones de los barcos:
 - 1 barco de 2 casillas
 - 2 barcos de 3 casillas
 - 1 barco de 4 casillas
 - 1 barco de 5 casillas

- En ningún caso podrá colocarse un barco en posiciones no contiguas ni en diagonal.

Localización:

- Archivo: Tablero.java
- Clase: Tablero
- Métodos:
 - colocarBarco(ArrayList<Coordenada> coordenadas, int dimensionBarco)
 - comprobarSolaparBarco(ArrayList<Coordenada> coordenadas)
 - comprobarBarcoDentroTablero(ArrayList<Coordenada> coordenadas)
 - comprobarCoordenadasContiguas(ArrayList<Coordenada> coordenadas, int dimensionBarco)

Test:

- Archivo: TableroTest.java
- Clase: TableroTest
- Métodos de Test asociado a la funcionalidad:
 - testColocarBarco2Casillas_DentroTablero_expectedTrue()
 - testColocarBarco2Casillas_FueraTableroHorizontal_expectedFalse()
 - testColocarBarco2Casillas_FueraTableroVertical_expectedFalse()
 - testColocarBarco5Casillas_DentroTablero_expectedTrue()
 - testColocarBarco5Casillas_FueraTableroHorizontal_expectedFalse()
 - testColocarBarco5Casillas_FueraTableroVertical_expectedFalse()
 - testColocarBarco2Casillas_expectedTrue()
 - testColocarBarco2Casillas_SolaparHorizontal_expectedTrue()
 - testColocarBarco2Casillas_SolaparVertical_expectedTrue()
 - testColocarBarco5Casillas_expectedTrue()
 - testColocarBarco5Casillas_SolaparHorizontal_expectedTrue()
 - testColocarBarco5Casillas_SolaparVertical_expectedTrue()
 - testComprobarCoordenadasContiguas_filaNoContigua_expectedFalse()
 - testComprobarCoordenadasContiguas_ColumnaNoContigua_expectedFalse()
 - testComprobarCoordenadasContiguas_diagonal_expectedFalse()
 - testComprobarCoordenadasContiguas_expectedTrue()

Para la realización de estos tests hemos tenido en cuenta las siguientes particiones equivalentes:

- Coordenadas válidas: coordenadas dentro del tablero, que no solapan con ningún barco ya colocado y son contiguas.
- Coordenadas no válidas: coordenadas fuera del tablero y/o que no solapan con ningún barco ya colocado y/o son contiguas.

Como valores límite y frontera, hemos tomado las 4 esquinas del tablero tanto dentro como fuera del mismo. Puede verse con más detalle en el código proporcionado.

2.5.3. Recibir golpe

Funcionalidad: cuando el tablero recibe un golpe debe asegurarse de buscar la casilla que se pretende golpear y tras realizar las oportunas comprobaciones, marcar dicha casilla como golpeada.

Localización:

- Archivo: Tablero.java
- Clase: Tablero
- Métodos:
 - recibirGolpe(Coordenada coordenada)
 - buscarCasilla(Coordenada coordenada)

Test:

- Archivo: TableroTest.java
- Clase: TableroTest
- Métodos de Test asociado a la funcionalidad:
 - testBuscarCasilla_Agua_expectedNotNullAndEquals()
 - testBuscarCasilla_expectedNotNullAndNotEquals()
 - testRecibirGolpe_casillaExiste_expectedTrue();
 - testRecibirGolpe_casillaNoExiste_expectedFalse();
 - testRecibirGolpe_casillaNull_expectedFalse();
 - testRecibirGolpe_casillaYaGolpeada_expectedFalse();

Para la realización de estos tests hemos tenido en cuenta las siguientes particiones equivalentes:

- Casilla es null.
- Casilla no existe (coordenada fuera de rango del tablero).
- Casilla existe (coordenada dentro del rango de tablero).
- Casilla existe, pero ya ha sido golpeada.

2.5.4. Comprobación barcos hundidos

Funcionalidad: implementación de la función comprobarTodosBarcosHundidos() que devolverá true en caso de que todos los barcos del tablero ya hayan sido golpeados.

Localización:

- Archivo: Tablero.java
- Clase: Tablero
- Métodos:
 - comprobarTodosBarcosHundidos()

Test: Para la realización de estos tests se ha tenido en cuenta que encontrara los barcos hundidos tanto si solo se había colocado 1 barco como si se había colocado 5 barcos.

- Archivo: TableroTest.java
- Clase: TableroTest
- Métodos de Test asociado a la funcionalidad:
 - testTodosBarcosHundidos1Barcos_expectedTrue()
 - testTodosBarcosHundidos5Barcos_expectedTrue()
 - testTodosBarcosHundidos1Barcos_expectedFalse()
 - testTodosBarcosHundidos5Barcos_expectedFalse()

2.6. Clase Jugador

Funcionalidad: Creación de la clase Jugador que representa a un jugador en el juego. Contiene atributos para identificar su nombre (`private String nombre`), y dos tableros (`private Tablero tableroPrincipal` y `private Tablero tableroSecundario`) que gestionan las posiciones de los barcos colocados previamente y los golpes recibidos mostrados como un segundo tablero. Los métodos permiten realizar operaciones como asignar un nombre, colocar barcos, recibir golpes y comprobar si todos los barcos han sido hundidos.

La primera versión de esta clase comenzó como una general que se iba a encargar de gestionar tanto al jugador de la consola como a la IA. Después de crear la clase Tablero, se decidió cambiar esta clase y declararla como abstracta (esto se ve reflejado en la rama JugadorV2), de manera que ahora tendría como subclases JugadorPersona y JugadorIA. Esto se hizo así principalmente porque los métodos que gestionaban la colocación de los barcos eran totalmente diferentes en los dos casos. La IA tendría que poner los barcos aleatoriamente mientras que el jugador lo entraría por consola.

Localización:

- Archivo: Jugador.java
- Clase: Jugador
- Métodos:
 - void Jugador(String nombre)
 - asignarNombre(String nombre)
 - getNombre()
 - asignarTablerosVacios(int dimension)
 - obtenerTableroPrincipal()
 - obtenerTableroSecundario()
 - comprobarTodosBarcosHundidos()
 - colocarBarco(ArrayList<Coordenada> casillasBarco, int dimensionBarco)
 - boolean recibirGolpe(Coordenada coordenada)
 - registrarGolpe(Coordenada coordenada, Tablero tableroPrincipalOponente)
 - adaptarTableroSecundario(Coordenada coordenada, Tablero tableroPrincipalOponente)

Test: Debido a que la primera versión de Jugador no era abstracta, se hicieron tests para comprobar que los métodos básicos en ese momento funcionaran.

- Archivo: JugadorTest.java

- Clase: JugadorTest
- Métodos de Test asociados a la funcionalidad:
 - testAsignarNombre_expectedTrue()
 - testObtenerTableroPrincipal_expectedTrue()
 - testObtenerTableroSecundario_expectedTrue()
 - testColocarBarcos_expectedTrue()
 - testColocarBarcos_expectedFalse()
 - testComprobarBarcosHundidos_expectedTrue()
 - testComprobarBarcosHundidos_expectedFalse()

2.6.1. Mock

En estos tests, se ha utilizado una clase Mock de Tablero para simular la funcionalidad del tablero, ya que su implementación no estaba acabada. Esto asegura que los métodos probados en JugadorTest puedan funcionar, independientemente de la clase Tablero.

Localización:

- Archivo: MockTablero.java
- Clase: MockTablero
- Métodos:
 - MockTablero(int dimension, boolean resultado)
 - colocarBarco(ArrayList<Coordenada> coordenadas)
 - comprobarTodosBarcosHundidos()

2.7. Clase JugadorIA

Funcionalidad: Creación de la clase JugadorIA que hereda de la clase abstracta Jugador. Representa a la IA que es el oponente del jugador. A diferencia del jugador, la IA debe generar las coordenadas de forma aleatoria, tanto para colocar los barcos como para golpear al tablero del jugador. A continuación se mostrarán los métodos implementados y los test correspondientes.

2.7.1. Métodos generales

Localización:

- Archivo: JugadorIA.java
- Clase: JugadorIA
- Métodos:
 - JugadorIA(String nombre, Random random)
 - JugadorIA(String nombre)
 - recibirGolpe(Coordenada coordenada)
 - void Jugador(String nombre)
 - asignarNombre(String nombre)
 - getNombre()
 - asignarTablerosVacios(int dimension)
 - obtenerTableroPrincipal()
 - obtenerTableroSecundario()

- comprobarTodosBarcosHundidos()
- registrarGolpe(Coordenada coordenada, Tablero tableroPrincipalOponente)
- adaptarTableroSecundario(Coordenada coordenada, Tablero tableroPrincipalOponente)

Test:

- Archivo: JugadorIATest.java
- Clase: JugadorIATest
- Métodos:
 - testRecibirGolpeValorFrontera0_ExpectedTrue();
 - testRecibirGolpeValorFrontera9_ExpectedTrue();
 - testRecibirGolpeValorLimite10_ExpectedFalse();
 - testRecibirGolpeValorLimiteNegativo_ExpectedFalse();
 - testRegistrarGolpe_casilla_Agua_expectedTrue();
 - testRegistrarGolpe_casilla_Barco_expectedTrue();
 - testAdaptarTableroSecundario_casillaBarco_expectedTrue();
 - testAdaptarTableroSecundario_casillaAgua_expectedTrue();
 - testAdaptarTableroSecundario_casillasGolpeadas_expectedTrue();
 - testAdaptarTableroSecundario_1barco_expectedFalse();
 - testAdaptarTableroSecundario_1barco_expectedTrue();
 - testAdaptarTableroSecundario_5barcos_expectedFalse();
 - testAdaptarTableroSecundario_5barcos_expectedTrue();

Para la realización de estos tests hemos tenido en cuenta en recibir golpe dos particiones equivalentes:

- Coordenada válida: dentro de rango
- Coordenada no válida: fuera de rango

Asimismo, hemos testado los valores frontera 0 y 9, y los valores límite -1 y 10. Para el método recibirGolpe hemos tenido que el golpe puede ser tanto en una casilla Agua como en una casilla Barco. Por último, para el test de adaptarTableroSecundario, hemos probado tanto para 1 barco como para 5 barcos.

2.7.2. Colocación de los barcos**Localización:**

- Archivo: JugadorIA.java
- Clase: JugadorIA
- Métodos:
 - colocarBarco(ArrayList<Coordenada> casillasBarco, int dimensionBarco)
 - generarCoordenadaAleatoria()
 - generarDireccionAleatoria()
 - calcularCoordenadasBarco(int fila, int col, int direccion, int dimensionBarco)

Test:

- Archivo: TestJugadorIA.java
- Clase: TestJugadorIA
- Métodos:
 - testColocarBarcoVerticalArriba2Dimensiones_ExpectedTrue()
 - testColocarBarcoVerticalAbajo2Dimensiones_ExpectedTrue()
 - testColocarBarcoHorizontalIzquierda2Dimensiones_ExpectedTrue()
 - testColocarBarcoHorizontalDerecha2Dimensiones_ExpectedTrue()
 - testColocarBarcoVerticalArriba2Dimensiones_ExpectedFalse()
 - testColocarBarcoVerticalAbajo2Dimensiones_ExpectedFalse()
 - testColocarBarcoHorizontalIzquierda2Dimensiones_ExpectedFalse()
 - testColocarBarcoHorizontalDerecha2Dimensiones_ExpectedFalse()
 - testColocarBarcoVerticalArriba5Dimensiones_ExpectedTrue()
 - testColocarBarcoVerticalAbajo5Dimensiones_ExpectedTrue()
 - testColocarBarcoHorizontalIzquierda5Dimensiones_ExpectedTrue()
 - testColocarBarcoHorizontalDerecha5Dimensiones_ExpectedTrue()
 - testColocarBarcoVerticalArriba5Dimensiones_ExpectedFalse()
 - testColocarBarcoVerticalAbajo5Dimensiones_ExpectedFalse()
 - testColocarBarcoHorizontalIzquierda5Dimensiones_ExpectedFalse()
 - testColocarBarcoHorizontalDerecha5Dimensiones_ExpectedFalse()

Para testear estos métodos, todos relacionados con la colocación de los barcos por parte del jugadorIA, hemos tenido en cuenta lo siguiente:

- Particiones equivalentes:
 - Coordenadas dentro de rango (barco de 2 y 5 coordenadas)
 - Coordenadas fuera de rango (barco de 2 y 5 coordenadas)
- Valores frontera dentro de rango: 4 esquinas del tablero
- Valores frontera dentro de rango: 4 esquinas del tablero con una coordenada fuera del mismo.

2.7.3. Gestión del golpe de las casillas**Localización:**

- Archivo: JugadorIA.java
- Clase: JugadorIA
- Métodos:
 - generarCoordenadaAleatoria()
 - golpear()

Test:

- Archivo: TestJugadorIA.java
- Clase: TestJugadorIA
- Métodos:
 - testGolpearCoordenadaExistenteValorFrontera0_ExpectedTrue()

- testGolpearCoordenadaExistenteValorFrontera9_ExpectedTrue()
- testGolpearCoordenadaNoExistenteMenorLimite_ExpectedTrue()
- testGolpearCoordenadaNoExistenteMayorLimite_ExpectedTrue()

Para las pruebas unitarias de esta funcionalidad, hemos tenido en cuenta lo siguiente:

- Particiones equivalentes:
 - Coordenada existente
 - Coordenadas no existente
- Valores frontera para coordenada existente:
 - Coordenada (0,0)
 - Coordenada(9,0)
- Valores limite para coordenada existente:
 - Coordenada(-1,-1)
 - Coordenada(10,10)

2.7.4. Mock

Para poder realizar los tests de la clase JugadorIA hemos decidido hacer un Mock de Random, pues como se ha expuesto anteriormente, JugadorIA genera coordenadas aleatorias tanto para colocar los barcos como para golpear una coordenada del oponente. Asimismo, también debe generar una dirección aleatoria sobre en qué dirección colocar los barcos en el Tablero. Por ello, necesitamos un Mock que nos permita devolver los valores esperados para los tests.

Localización:

- Archivo: MockRandom.java
- Clase: MockRandom
- Métodos:
 - MockRandom(int[] valores)
 - nextInt(int valorMaximo)

2.8. Clase JugadorPersona

Funcionalidad: Se ha creado esta clase para los métodos que usará el jugador desde la consola. Se sobrescribieron los métodos colocarBarco(ArrayList<Coordenada> casillasBarco), recibirGolpe(Coordenada coordenada), registrarGolpe(Coordenada coordenada, Tablero tableroPrincipalOponente) para poder actualizar los tableros que utiliza (TableroPrincipal y TableroSecundario).

2.8.1. Métodos generales

Localización:

- Archivo: JugadorPersona.java
- Clase: JugadorPersona
- Métodos:
 - JugadorPersona(String nombre)
 - recibirGolpe(Coordenada coordenada)

- void Jugador(String nombre)
- asignarNombre(String nombre)
- getNombre()
- asignarTablerosVacios(int dimension)
- obtenerTableroPrincipal()
- obtenerTableroSecundario()
- comprobarTodosBarcosHundidos()
- registrarGolpe(Coordenada coordenada, Tablero tableroPrincipalOponente)
- adaptarTableroSecundario(Coordenada coordenada, Tablero tableroPrincipalOponente)

Test: Se han reutilizado los tests de la superclase Jugador, pero ahora cambiando la declaración de una instancia Jugador con JugadorPersona. Además, en el momento de crear JugadorPersona la clase Tablero ya estaba acabado, permitiendo cambiar las llamadas de MockTablero a Tablero directamente.

- Archivo: JugadorTest.java
- Clase: JugadorTest
- Métodos:
 - testAsignarNombre_expectedTrue()
 - recibirGolpe_expectedTrue()
 - recibirGolpe_expectedFalse()
 - testObtenerTableroPrincipal_expectedTrue()
 - testObtenerTableroSecundario_expectedTrue()
 - testComprobarBarcosHundidos_expectedTrue()
 - testComprobarBarcosHundidos_expectedFalse()
 - testRegistrarGolpe_casillaBarco_expectedTrue()
 - testRegistrarGolpe_casillaAgua_expectedTrue()
 - testAdaptarTableroSecundario_casillaBarco_expectedTrue()
 - testAdaptarTableroSecundario_casillaAgua_expectedTrue()
 - testAdaptarTableroSecundario_casillasGolpeadas_expectedTrue()

La implementación de los métodos testeados es prácticamente igual que en JugadorIA, las dos clases derivan de la clase abstracta Jugador. Únicamente hay la diferencia de que las llamadas al oponente son a jugadorIA. Por tanto, los tests realizados y la lógica seguida es la misma que la ya explicada en el apartado anterior, JugadorIA métodos generales.

2.8.2. Colocación de barcos

Localización:

- Archivo: JugadorPersona.java
- Clase: JugadorPersona
- Métodos:
 - colocarBarco(ArrayList<Coordenada> casillasBarco, int dimensionBarco)

Test:

- Archivo: JugadorTest.java
- Clase: JugadorTest
- Métodos:
 - testColocarBarcos_2casillas_dentroTablero_esquinaSuplIzq_expectedTrue()
 - testColocarBarcos_2casillas_dentroTablero_esquinaInfIzq_expectedTrue()
 - testColocarBarcos_2casillas_dentroTablero_esquinaSupDer_expectedTrue()
 - testColocarBarcos_2casillas_dentroTablero_esquinaInfDer_expectedTrue()
 - testColocarBarcos_2casillas_dentroTablero_esquinaSuplIzq_expectedFalse()
 - testColocarBarcos_2casillas_dentroTablero_esquinaInfIzq_expectedFalse()
 - testColocarBarcos_2casillas_dentroTablero_esquinaSupDer_expectedFalse()
 - testColocarBarcos_2casillas_fueraTablero_esquinaInfDer_expectedFalse()

Para la realización de estos test hemos tenido en cuenta lo siguiente:

- Particiones equivalentes:
 - Coordenadas válidas: dentro de rango
 - Coordenadas no válidas: fuera de rango
- Valores frontera: las 4 esquinas del tablero
- Valores límite: una de las coordenadas del barco se encuentra fuera de rango

2.9. Clase Partida

Funcionalidad:

La clase Partida se encarga de gestionar los eventos que ocurren durante el juego. Esto son, crear el tablero, colocar los barcos de los dos jugadores en sus respectivos tableros, llamar a los métodos correspondientes para golpear las casillas que se seleccionen en el tablero, gestionar los turnos y la comprobación del final de la partida.

Localización:

- Archivo: Partida.java
- Clase: Partida
- Métodos:
 - Partida(Jugador jugadorPersona, Jugador jugadorIA)
 - getJugadorPersona()
 - getJugadorIA()
 - getDimensionTablero()
 - setDimensionTablero(int dimension)
 - crearTablero(int dimension)
 - colocarBarcoJugador(ArrayList<Coordenada> coordenadas)
 - colocarBarcosIA()
 - golpearJugadorPersona(Coordenada coordenada)
 - golpearJugadorIA()
 - obtenerTurno()
 - cambiarTurno()

- comprobarFinPartida()

Tests:

- Archivo: Partida.java
- Clase: Partida
- Métodos:
 - constructorPartida_expectedTrue()
 - testGetDimensionTablero_expectedTrue()
 - colocarBarcoJugador_expectedTrue()
 - colocarBarcoJugador_expectedFalse()
 - colocarBarcosIA_verifyMethod()
 - golpeaJugadorPersona_expectedTrue()
 - golpeaJugadorPersona_expectedFalse()
 - golpeaJugadorIA_expectedTrue()
 - golpeaJugadorIA_expectedFalse()
 - cambiarTurno_primerCambio_turno2_expectedTrue()
 - cambiarTurno_segundoCambio_turno1_expectedTrue()
 - comprobarFinPartida1_expectedTrue()
 - comprobarFinPartida2_expectedTrue()
 - comprobarFinPartida3_expectedTrue()
 - comprobarFinPartida4_expectedFalse()

2.9.1. Mockito

Para los tests de la clase Partida se ha utilizado Mockito para JugadorIA y JugadorPersona, ya que los métodos de Partida llamaban a estas clases. Los dos jugadores deben colocar sus barcos, cada uno de una manera diferente, por lo tanto, estos valores no se pueden determinar de por sí solos, era necesario especificar que devolvería cada uno.

3. Controlador

Funcionalidad: La clase Controlador interactúa con la vista (VistaConsola) y el modelo (con las clases principales Partida, JugadorPersona y JugadorIA). Su responsabilidad es gestionar el flujo del juego, inicializando la partida y revisando si llega el final, esto se da si alguno de los dos jugadores hunde todos los barcos del contrincante. Para esto, se han creado métodos para validar los caracteres de las coordenadas que escribe el jugador desde la consola, la colocación de barcos llamando a la clase partida y la alternancia de turnos entre los dos jugadores.

Localización:

- Archivo: Controlador.java
- Clase: Controlador
- Métodos:
 - Controlador(VistaConsola vista, Partida partida, JugadorPersona jugadorPersona, JugadorIA jugadorIA)
 - Partida getPartida()
 - getVistaConsola()

- `getJugadorPersona()`
- `getJugadorIA()`
- `comprobarFormatoCoordenadas(String coordenadasJugador, int casillasTotales)`
- `validarLongitudCoordenada(String coordenada)`
- `validarCoordenada2Caracteres(char[] caracteres, int limiteFila, char limiteColumna)`
- `validarCoordenada3Caracteres(char[] caracteres, int limiteFila, char limiteColumna)`
- `parsearCoordenada(String coordenada)`
- `colocarBarcosJugadorPersona()`
- `comenzarPartida()`
- `turnoJugadorpersona()`
- `turnoJugadorIA()`

Test:

- Archivo: `ControladorTest.java`
- Clase: `ControladorTest`
- Métodos de Test asociado a la funcionalidad:
 - `constructorControlador_expectedTrue()`
 - `testParsearCoordenada_2Caracteres_expectedTrue()`
 - `testParsearCoordenada_3Caracteres_expectedTrue()`
 - `testColocarBarcosJugadorPersona_expectedTrue()`
 - `comenzarPartida_NoFinPartida_TurnoJugador1()`
 - `comenzarPartida_NoFinPartida_TurnoJugador2()`
 - `comenzarPartida_FinPartida()`

3.1. Mockito

Se ha utilizado Mockito para simular la interacción con las clases `Vista`, `Partida`, `JugadorIA` y `JugadorPersona`. Esto es así debido a que `Controlador` llama a los métodos de dichas clases y espera ciertos valores. Por esto, es necesario especificar que valores se devuelven con tal de ejecutar los tests de `Controlador` correctamente.

3.2. Pairwise testing

Para la comprobación del formato de coordenadas que pasa el jugador se ha realizado un *pairwise testing* para que se cubran todos los casos de posibles errores. Se han hecho teniendo en cuenta las siguientes particiones equivalentes y valores frontera/límite:

- Particiones equivalentes:
 - Válido: El string coincide con el formato esperado.
 - No válido: El string no coincide con el formato esperado.
- Valores frontera:
 - Mínimo: 0 (para las filas) y "A" (para las columnas).

- Máximo: Dimensión del tablero - 1 (para las filas) y la última letra posible en el tablero (según la dimensión).

Caso	Dimensión del tablero	Tamaño del barco	Partición equivalente (Válida/No válida) Coordenadas (String)
1	10	2	Válida: 0A, 0B
2	10	5	Válida: 0A, 0B, 0C, 0D, 0E
3	15	2	Válida: 14A, 14B
4	15	5	Válida: 14A, 14B, 14C, 14D, 14E
5	10	2	No válida: 10A, 10B
6	10	5	No válida: 0P, 0Q, 0R, 0S, 0T
7	15	2	No válida: 14A
8	15	5	No válida: 14A14B14C14D14E
9	10	2	No válida: A0, B0
10	15	5	No válida: 0A, 0B, 0C, 0D, 0E

Tests asociados a los casos:

- testComprobarFormatoCoordenadas_dimTablero10_barco2casillas_letraAntesQueNumero_expectedFalse()
- testComprobarFormatoCoordenadas_dimTablero15_barco5casillas_coordenadasPequeñas_expectedFalse()
- testComprobarFormatoCoordenadas_dimTablero15_barco5casillas_sinEspacios_expectedFalse()
- testComprobarFormatoCoordenadas_dimTablero15_barco2casillas_cantidadIncorrecta_expectedFalse()
- testComprobarFormatoCoordenadas_dimTablero10_barco5casillas_letraFueraDeRango_expectedFalse()
- testComprobarFormatoCoordenadas_dimTablero10_barco2casillas_numeroFueraDeRango_expectedFalse()
- testComprobarFormatoCoordenadas_dimTablero15_barco5casillas_expectedTrue()
- testComprobarFormatoCoordenadas_dimTablero15_barco2casillas_expectedTrue()
- testComprobarFormatoCoordenadas_dimTablero10_barco5casillas_expectedTrue()
- testComprobarFormatoCoordenadas_dimTablero10_barco2casillas_expectedTrue()

4. Vista

Funcionalidad: La clase VistaConsola es la encargada de mostrar todos los resultados o mensajes por la consola mientras la partida esté activa, así como la encargada de pedir la información necesaria al jugador. Esto engloba los mensajes para pedir inputs, los mensajes de errores durante el juego o información e imprimir el tablero junto con las casillas correctas.

Localización:

- Archivo: VistaConsola.java
- Clase: VistaConsola
- Métodos:
 - VistaConsola()
 - setPartida(Partida partida)
 - mostrarMensajeInicioJuego()
 - mostrarMensajeFinJuego()
 - mostrarMensajeTurnoIA()
 - pedirDimensionTablero()
 - convertirDimensionTablero(int opcionSeleccionada)
 - pedirColocarBarco(int dimensionBarco)
 - pedirColocarBarco2casilla()
 - pedirColocarBarco3casilla()
 - pedirColocarBarco4casilla()
 - pedirColocarBarco5casilla()
 - pedirGolpe()
 - mostrarErrorCoordenada()
 - mostrarTableros()
 - printTableros(ArrayList<Casilla> tableroPrincipal, ArrayList<Casilla> tableroSecundario, int filas, int columnas)

Test: Dado que esta clase es la encargada de imprimir los resultados por la consola, no es necesario ningún Test.

5. Pruebas de caja blanca

5.1. Statement coverage

A continuación se muestra una captura de pantalla donde se demuestra que cumplimos el statement coverage al 100%.

Coverage Juego x			
Element	Class, %	Method, %	Line, %
all	100% (11/11)	100% (79/79)	100% (298/298)
Modelo	100% (10/10)	100% (65/65)	100% (203/203)
JugadorIA	100% (1/1)	100% (8/8)	100% (40/40)
Tablero	100% (1/1)	100% (12/12)	100% (67/67)
MockRandom	100% (1/1)	100% (2/2)	100% (6/6)
Coordenada	100% (1/1)	100% (6/6)	100% (11/11)
Jugador	100% (1/1)	100% (9/9)	100% (17/17)
Casilla	100% (1/1)	100% (6/6)	100% (14/14)
JugadorPersona	100% (1/1)	100% (3/3)	100% (3/3)
Partida	100% (1/1)	100% (13/13)	100% (35/35)
Agua	100% (1/1)	100% (3/3)	100% (5/5)
Barco	100% (1/1)	100% (3/3)	100% (5/5)
Controlador	100% (1/1)	100% (14/14)	100% (95/95)
Controlador	100% (1/1)	100% (14/14)	100% (95/95)

5.2. Decision coverage

A continuación se mostrarán capturas de 3 métodos distintos que cumplen con decision coverage.

- Método `cambiarTurno()` de la clase `Partida` (`Partida.java`):

```

72  public void cambiarTurno() { 7 usages
73      if (turno == 1)
74          turno = 2;
75      else
76          turno = 1;
77  }

```

- Método `comenzarPartida()` de la clase `Controlador` (`Controlador.java`):

```

147  public void comenzarPartida() { 6 usages
148      while (!partida.comprovarFinPartida()) {
149          if (partida.obtenerTurno() == 1) {
150              turnoJugadorpersona();
151          } else {
152              turnoJugadorIA();
153          }
154          partida.cambiarTurno();
155      }
156      vista.mostrarMensajeFinJuego();
157  }

```

- Método `colocarBarco()` de la clase `JugadorIA(JugadorIA.java)`:

```

20 public boolean colocarBarco(ArrayList<Coordenada> casillasBarco, int dimensionBarco) {
21     boolean barcoColocado;
22     int fila = generarCoordenadaAleatoria();
23     int col = generarCoordenadaAleatoria();
24     int direccion = generarDireccionAleatoria();
25     ArrayList<Coordenada> coordenadas = calcularCoordenadasBarco(fila, col, direccion, dimensionBarco);
26     if (coordenadas != null) {
27         barcoColocado = tableroPrincipal.colocarBarco(coordenadas, dimensionBarco);
28     } else {
29         barcoColocado = false;
30     }
31     return barcoColocado;
32 }

```

5.3. Condition coverage

A continuación se mostrarán capturas de 3 métodos distintos que cumplen con condition coverage.

- Función `comprobarFinPartida()` de la clase `Partida (Partida.java)`:

```

79 public boolean comprobarFinPartida() { 15 usages
80     boolean jugador1Perdido = jugadorPersona.comprobarTodosBarcosHundidos();
81     boolean jugador2Perdido = jugadorIA.comprobarTodosBarcosHundidos();
82     if (jugador1Perdido || jugador2Perdido) {
83         return true;
84     }
85     return false;
86 }
87 }

```

- Función `recibirGolpe()` de la clase `Tablero(Tablero.java)`:

```

104 public boolean recibirGolpe(Coordenada coordenada) {
105     Casilla casillaGolpeada = buscarCasilla(coordenada);
106
107     if (casillaGolpeada != null && !casillaGolpeada.esGolpeada()) {
108         casillaGolpeada.recibirGolpe();
109         return true;
110     }
111     return false;
112 }

```

- Función `comprobarTodosBarcosHundidos()` de la clase `Tablero(Tablero.java)`:

```

124 public boolean comprobarTodosBarcosHundidos() { 5 usages
125     for (Casilla casilla : tablero) {
126         if (casilla.getId() != 0 && !casilla.esGolpeada()) {
127             return false;
128         }
129     }
130     return true;
131 }
132 }

```

5.4. Path coverage

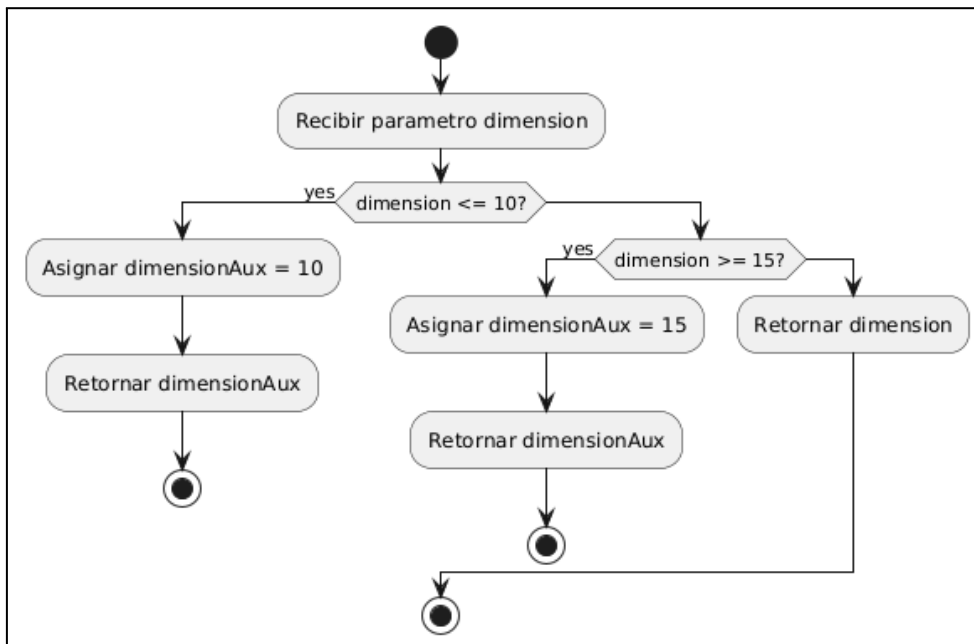
A continuación se mostrarán capturas de 2 métodos distintos que cumplen con path coverage y el correspondiente diagrama de actividades en UML.

- Función comprobarDimension() de la clase Tablero(Tablero.java)

```

22     public int comprobarDimension(int dimension) { 1 usage
23         int dimensionAux;
24         if (dimension <= 10) {
25             dimensionAux = 10;
26             return dimensionAux;
27         } else if (dimension >= 15) {
28             dimensionAux = 15;
29             return dimensionAux;
30         }
31         return dimension;
32     }

```

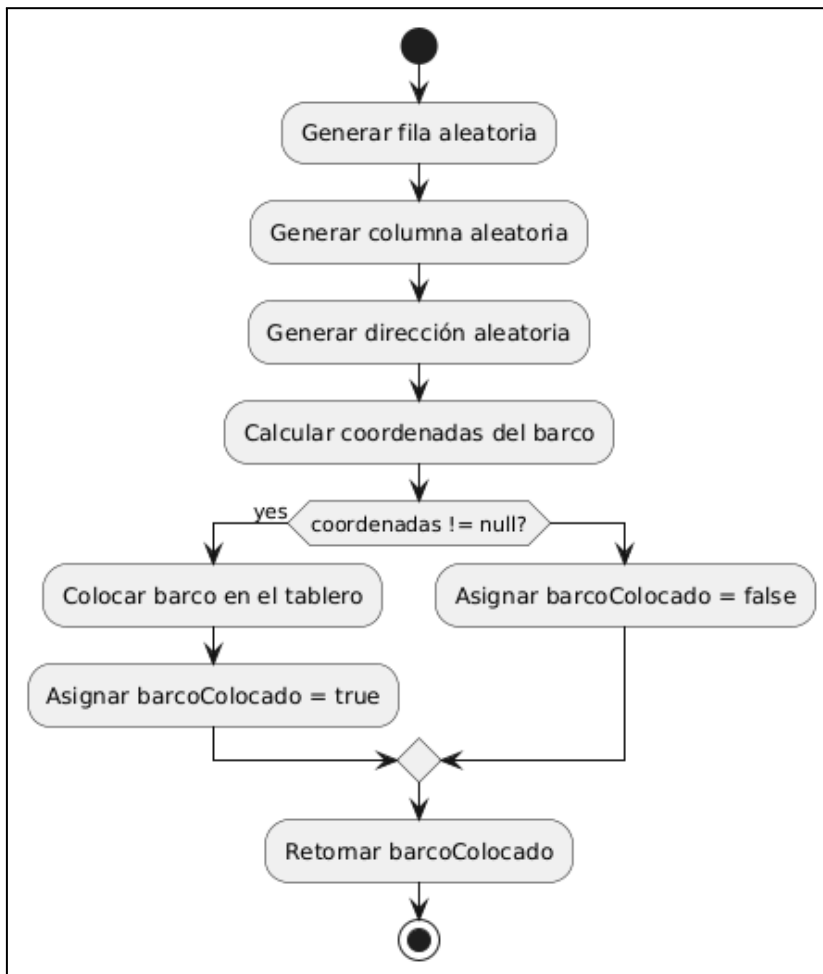


- Función colocarBarco de la clase JugadorIA(JugadorIA.java)

```

19     @Override 59 usages
20     public boolean colocarBarco(ArrayList<Coordenada> casillasBarco, int dimensionBarco) {
21         boolean barcoColocado;
22         int fila = generarCoordenadaAleatoria();
23         int col = generarCoordenadaAleatoria();
24         int direccion = generarDireccionAleatoria();
25         ArrayList<Coordenada> coordenadas = calcularCoordenadasBarco(fila, col, direccion, dimensionBarco);
26         if (coordenadas != null) {
27             barcoColocado = tableroPrincipal.colocarBarco(coordenadas, dimensionBarco);
28         } else {
29             barcoColocado = false;
30         }
31         return barcoColocado;
32     }

```

5.5. Loop Testing

Hemos utilizado una función del método tablero para hacer loop testing. Se pueden encontrar los tests en TableroTest.java. A continuación se muestra una imagen de la función que contiene el loop que ha sido testeado con loop testing.

```

80 @ public boolean comprobarBarcoDentroTablero(ArrayList<Coordenada> coordenadas) {
81     // Verificar si todas las coordenadas del barco están dentro del tablero
82     for (Coordenada coordenada : coordenadas) {
83         if (coordenada.getFila() >= numFilas || coordenada.getFila() < 0 ||
84             coordenada.getCol() >= numCols || coordenada.getCol() < 0) {
85             return false;
86         }
87     }
88     return true;
89 }
  
```

Para realizar el loop testing hemos tenido en cuenta que la dimensión máxima de un barco es de 5 coordenadas. Teniendo esto presente hemos realizado los siguientes tests.

- 0 pasadas: testComprobarBarcoDentroTablero_LoopTesting_0pasadas_ExpectedTrue()
- 1 pasadas: testComprobarBarcoDentroTablero_LoopTesting_1pasada_ExpectedTrue()
- 2 pasadas: testComprobarBarcoDentroTablero_LoopTesting_2pasadas_ExpectedTrue()

- $M < n$ pasadas (siendo $n = 5$):
`testComprobarBarcoDentroTablero_LoopTesting_NMenorQueNpasadas_ExpectedTrue()`
- $n - 1$ pasadas:
`testComprobarBarcoDentroTablero_LoopTesting_NMenorQueNpasadas_ExpectedTrue()`
- n pasadas: `testComprobarBarcoDentroTablero_LoopTesting_Npasadas_ExpectedTrue()`

6. CI/CD

Funcionalidad: Se crearon los archivos yaml en la correspondiente carpeta para activar Actions de GitHub. Cabe decir que después de realizar varias pruebas, en el momento de ejecutar los archivos en GitHub, hay errores de compilación externos a los archivos.

Localización:

- Archivo para checkstyle: `.github/workflows/checkstyle.yml`
- Archivo para tests: `.github/workflows/tests.yml`